

Digital Systems & Microcontrollers

Endsem Answer Key

1st December, 2025

Question-1

Load two numbers in registers R0 and R1. For instance 18 and 4 and check if the contents of R0, [R0] is divisible by [R1]. If divisible store 1 at memory location 100 else store 0 at the same memory location 100. Assume that the program is stored from memory location 0. Clearly show the address of the memory location and the code/data stored at that location.

- (a) Bonus of 5 points if you can do the above code by recursion and trace the contents of Stack Pointer and Program Counter for every function and return calls.
- (b) Note: As mentioned in Section 6.1.4, Flags and Program Status Word of the Processor Design pdf, the Sign Flag, S is set if an Accumulator operation results in a negative number.

[10 marks + 5 marks bonus]

Solution:

The problem requires writing an assembly program to check if a number in register R0 is divisible by a number in R1. The result, 1 for divisible or 0 for not divisible, should be stored at memory location 100. Similar to what we did in the lectures for multiplication (repeated addition), here the core logic relies on repeated subtraction.

We will present two solutions: an iterative method to fulfill the main requirement, and a recursive method for the bonus.

Part 1: Iterative Solution (Using a Loop)

This is the standard approach using a loop. The program repeatedly subtracts the divisor (R1) from the dividend (R0) and uses conditional jumps based on the processor's flags to give the outcome.

The complete assembly program is shown in the table below, with each instruction at its specified memory address.

Table 1: Assembly Language Program: Division by Subtraction

Phase	Addr	Code	Operation / Comment
Initialization	00	movi R0	Prepare to load dividend
	01	18	$R0 \leftarrow 18$ (Dividend)
	02	movi R1	Prepare to load divisor
	03	04	$R1 \leftarrow 4$ (Divisor)
Subtraction Loop	04 Loop:	movs R0	$[AR] \leftarrow [R0]$ (Move R0 to AR)
	05	sub R1	$[AR] \leftarrow [AR] - [R1]$ (Subtract Divisor)
	06	movd R0	$[R0] \leftarrow [AR]$ (Update R0)
	07	jumpdZ	If $Z = 1$ (Result 0), Jump to LoopZ
	08	LoopZ	Target Address: 13
	09	jumpdS	If $S = 1$ (Result -ve), Jump to LoopS
	10	LoopS	Target Address: 20
	11	jumpdU	Unconditional Jump (Continue Loop)
Case: Divisible (LoopZ)	12	Loop	Target Address: 04
	13 LoopZ:	movi R2	Prepare pointer for output
	14	100	Value: 100
	15	movs R2	$[AR] \leftarrow [R2]$ (Point AR to 100)
	16	movi R3	Prepare "True" flag
	17	01	$R3 \leftarrow 1$
	18	stor R3	$M[100] \leftarrow [R3]$ (Store 1)
	19	stop	Halt program
Case: Not Divisible (LoopS)	20 LoopS:	movi R2	Prepare pointer for output
	21	100	Value: 100
	22	movs R2	$[AR] \leftarrow [R2]$ (Point AR to 100)
	23	movi R3	Prepare "False" flag
	24	00	$R3 \leftarrow 0$
	25	stor R3	$M[100] \leftarrow [R3]$ (Store 0)
	26	stop	Halt program

Key Steps:

- **Initialization (Addr 00-03):** The dividend (18) is loaded into R0 and the divisor (4) into R1.
- **Subtraction Loop (Addr 04-12):** The value in R1 is subtracted from R0.
- **Conditional Checks:**

- **jumpZ 13:** If the subtraction results in zero, the **Zero Flag (Z)** is set. This means the numbers are perfectly divisible, and the program jumps to address 13.
- **jumpS 20:** If the result becomes negative, the **Sign Flag (S)** is set (as noted in the question). This means the numbers are not divisible, and the program jumps to address 20.
- **jumpU 04:** If neither flag is set, the loop continues.
- **Store Result (Addr 13-19 or 20-26):** The program stores 1 (for divisible) or 0 (for not divisible) at memory address 100 and then halts.

Part 2: Recursive Solution (Bonus)

For the bonus points, the problem is solved using a recursive function, which repeatedly calls itself. This method makes extensive use of the stack to store return addresses for each call.

Key Components:

- **Main Program (Addr 00-10):** This section loads the initial values, calls the recursive function `isDiv` for the first time, and, upon its final return, stores the result from R3 into memory location 100.
- **Recursive Function `isDiv` (Addr 20-37):**
 - **Base Cases:** It checks for a zero result (`jumpdZ`) to return 1 (in R3), or a negative result (`jumpdS`) to return 0 (in R3). These are the conditions that stop the recursion.
 - **Recursive Step:** If neither base case is met, it calls itself (`call 20`) to perform the next subtraction. This action pushes the return address (37) onto the stack.
 - **Return:** The `retu` instruction pops the return address from the stack, allowing the program to unwind from the recursive calls.

The assembly code for this approach is as follows:

Addr	Instruction	Comment / Operation
— Main Program —		
00	movi R0	Prepare to load Dividend
01	18	$R0 \leftarrow 18$
02	movi R1	Prepare to load Divisor
03	04	$R1 \leftarrow 4$
04	cdu	Call isDiv (Pushes RetAddr 06 to Stack)
05	isDiv	Target Address of function: 20
06	movi R2	Return point; Prepare pointer for output
07	100	Memory Location 100
08	movs R2	$[AR] \leftarrow 100$
09	stor R3	Store result ($R3$) to Memory[100]
10	stop	Halt Execution
— Recursive Function (isDiv) —		
20	movs R0	$[AR] \leftarrow [R0]$
21	sub R1	$[AR] \leftarrow [AR] - [R1]$
22	movd R0	$[R0] \leftarrow [AR]$ (Update Dividend)
23	jumpdZ	If $Z = 1$ (Result 0), Jump to Base Case LoopZ
24	LoopZ	Target Address: 29
25	jumpdS	If $S = 1$ (Result Negative), Jump to Base Case LoopS
26	LoopS	Target Address: 32
27	jumpdU	Else, Jump to Recursive Step
28	Loop	Target Address: 35 (Recursion)
— Base Cases —		
29	LoopZ: movi R3	Case: Divisible. Prepare True flag.
30	01	$R3 \leftarrow 1$
31	retu	Return from subroutine
32	LoopS: movi R3	Case: Not Divisible. Prepare False flag.
33	00	$R3 \leftarrow 0$
34	retu	Return from subroutine
— Recursive Call Block —		
35	Loop: cdu	Recursion. Call isDiv (Pushes RetAddr 37)
36	isDiv	Target: Start of function
37	retu	Return (Unwinds stack after recursive child returns)

Execution Trace for Recursive Solution (Bonus)

As required by the bonus question, the table below traces the state of the Program

Counter (PC), Stack Pointer (SP), and the value at the top of the stack ([SP]) through each step of the recursive execution for inputs 18 and 4. This clearly shows the stack building during the forward recursion and unwinding as the function returns.

Observe how the Stack Pointer (SP) decrements as we go deeper into recursion (calls at line 35 & 04) and increments as we return (line 37 or base cases).

Mark	Step	Operation / Context	[PC]	[R0]	[SP]	[[SP]]
<i>Phase 1: Forward Recursion (Building the Stack)</i>						
	1	Initial State	00	X	255	X
	2	Main: Load 18, 4	04	18	255	X
✓	3	1st Call to isDiv (from 04)	20	18	254	06
	4	Sub: $18 - 4 = 14$	27	14	254	06
✓	5	2nd Call to isDiv (Recursion at 35)	20	14	253	37
	6	Sub: $14 - 4 = 10$	27	10	253	37
✓	7	3rd Call to isDiv (Recursion at 35)	20	10	252	37
	8	Sub: $10 - 4 = 6$	27	6	252	37
✓	9	4th Call to isDiv (Recursion at 35)	20	6	251	37
	10	Sub: $6 - 4 = 2$	27	2	251	37
✓	11	5th Call to isDiv (Recursion at 35)	20	2	250	37
<i>Phase 2: Base Case Hit</i>						
	12	Sub: $2 - 4 = -2$ (Negative)	22	-2	250	37
	13	jumpdS triggers. Jump to 32 (LoopS).	32	-2	250	37
	14	Set $R3 = 0$ (False)	33	-2	250	37
<i>Phase 3: Unwinding the Stack (Returns)</i>						
✓	15	retu from LoopS (Return from 5th Call)	37	-2	251	37
✓	16	Exec 37: retu (Return from 4th Call)	37	-2	252	37
✓	17	Exec 37: retu (Return from 3rd Call)	37	-2	253	37
✓	18	Exec 37: retu (Return from 2nd Call)	37	-2	254	06
✓	19	Exec 37: retu (Return from 1st Call)	06	-2	255	X
<i>Phase 4: Completion</i>						
	20	Store Result: $M[100] \leftarrow 0$	09	-2	255	X
	21	Stop	10	-2	255	X

Note: Rows with checkmarks (✓) indicate the points where a function call or return occurs, showing the changes in the stack. Only these rows will be considered for grading. Rest of the rows are for clarity and understanding of the flow.

Question-2

Design a serial 2's complementer with a shift register (4 bit), logic and a flip-flop. The binary number is shifted out from one side and it's 2's complement shifted into the other side of the shift register. [10 marks]

Solution:

There are two common methods to compute the 2's complement of a binary number:

- **Method-1:** Invert all bits and add 1.
- **Method-2:** Copy all bits up to and including the first '1' (from the least significant bit side), then invert all remaining bits.

Actually, if you think carefully, Method-2 is the outcome from Method-1 only. But in terms of hardware implementation, Method-2 is easier and provides less hardware complexity in a serial circuit.

Method-1: The Serial Adder Method

Core Idea:

The 2's complement of a binary number is obtained by inverting all bits and adding 1.

$$2's\ Comp(N) = \bar{N} + 1$$

In a serial circuit, this can be achieved by using a half-bit adder to add 1 to the inverted bits as they are shifted out and propagate the carry backwards.

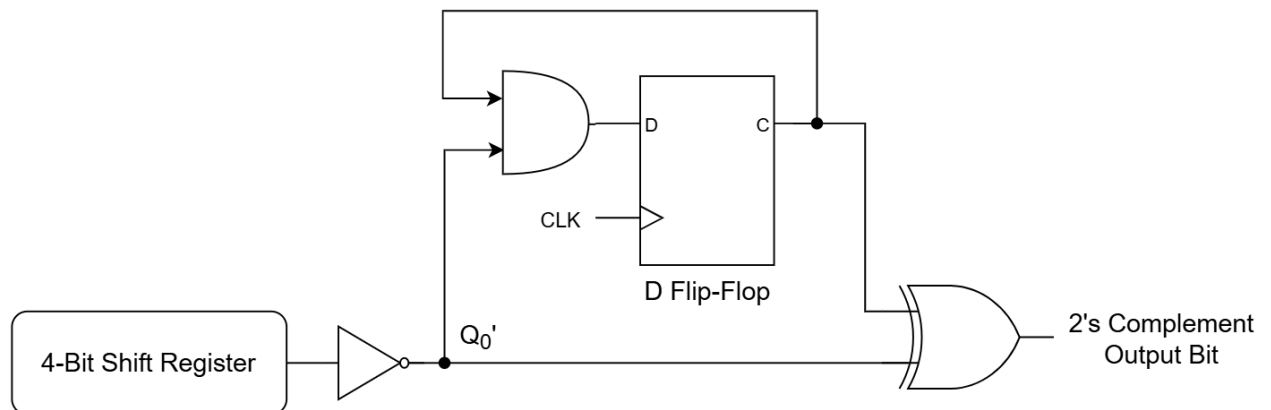
Hardware Design:

- **4-bit Shift Register:** Holds the input number. Shifts bits out serially (Q0) and shifts in the computed 2's complement bits.
- **Logic Circuit:** Inverts the bit being shifted out (Q0) and then adds the inverted bit from Q0 and the carry from the previous addition.
- **Flip-Flop (Carry):** Stores the carry bit from the half-bit adder for the next addition.

Equations

Here, we are using half-bit adder logic to add the inverted bit and carry. For a half-bit adder with inputs $A = \bar{x}$ and Carry C :

$$\begin{aligned} Sum(S) &= A \oplus C_{in} = \bar{x} \oplus C \\ Next\ Carry(D) &= A \cdot C_{in} = C \cdot \bar{x} \end{aligned}$$

Circuit Diagram:

Initial State: This method requires the Flip-Flop to be **Asynchronously Presets (Set to 1)** before the operation begins. This initial '1' represents the "+1" in the 2's complement formula.

Method-2: The First-One Detection Method

Core Idea: The 2's complement of a binary number is obtained by inverting all bits after the first '1' (from the least significant bit side) and keeping the rest unchanged.

So we have to think about designing logic that can detect when the first '1' has been shifted out. To track whether we have encountered the first '1' bit yet, we can use a flip-flop to store this state.

Hardware Design:

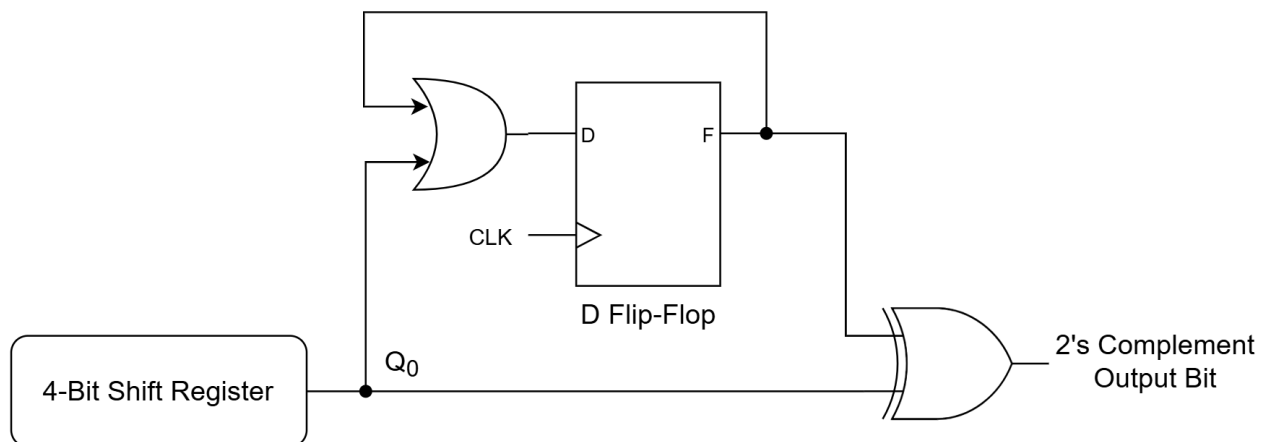
- **4-bit Shift Register:** Holds the input number. Shifts bits out serially (Q_0) and shifts in the computed 2's complement bits.
- **Flip-Flop (Flag):** Indicates whether the first '1' has been seen. Initialized to 0 before operation.
- **Logic Circuit:** Determines whether to invert the bit being shifted in, based on the flag.

Equations

Let Q_0 = current output bit from the register. Let F = flip-flop output (flag).

$$\text{Bit to shift in}(D_{in}) = \begin{cases} Q_0 & \text{if } F = 0 \\ \overline{Q_0} & \text{if } F = 1 \end{cases} \rightarrow \text{XOR}$$

$$\text{Next Flag}(F_{next}) = F + Q_0$$

Circuit Diagram:

Initial State: This method requires the Flip-Flop to be **Asynchronously Cleared (Reset to 0)** before the operation begins to assume the "First 1 not found" state.

The flip-flop must be reset to 0 before starting.

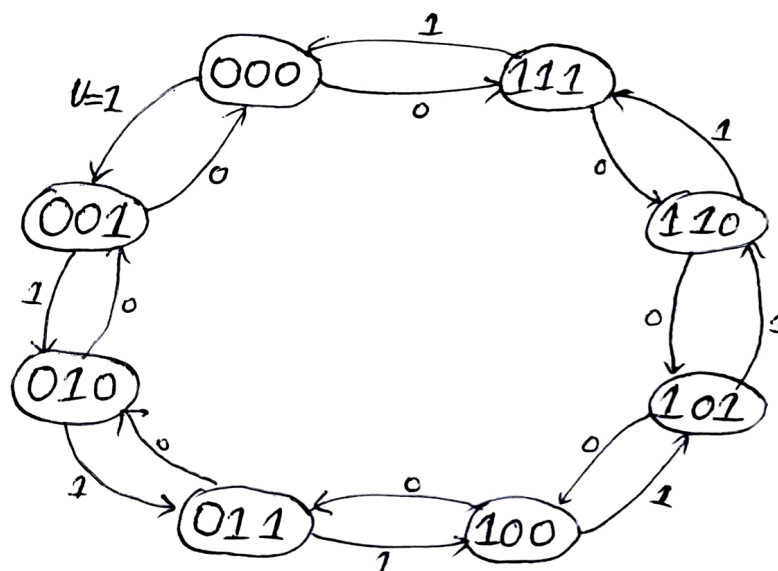
Question-3

Design a synchronous 3 bit UP/DOWN counter that counts UP till it equals the value specified in a 3 bit register B, after which it starts counting till zero after which it starts counting up again. Depict the Finite State Machine, mention clearly below the FSM what do the variables in the FSM stand for and from the FSM design the eventual sequential circuit. Please note that your method needs to work for any 3 bit register value and not just for a specific case. [10 marks]

Solution:

We define a state variable U representing the direction of the counter.

- **State $U = 1$ (UP):** The counter increments.
- **State $U = 0$ (DOWN):** The counter decrements.



State Diagram for 3-bit UP/DOWN Counter

The Value of Register B can be any 3-bit number from 000 to 111 as shown in the figure above.

So basically, as the question states, the counter counts UP until it reaches the value in Register B (signaled by $E = 1$), then switches to counting DOWN until it reaches zero (signaled by $Z = 1$), and then switches back to counting UP, and so on. So we have to update the state variable U based on these conditions which can be determine using following input signals:

- E : **Equal** signal. High (1) when Counter Q equals Register B .
- Z : **Zero** signal. High (1) when Counter Q equals 0.

Let 3-Bit numbers in Register B and Counter Q be represented as:

- Register B: $B = R_2R_1R_0$ (3-bit limit value)
- Counter Q: $Q = Q_2Q_1Q_0$ (3-bit current count)

Signal Equations:

- **Zero Detector:** $Z = \overline{Q_2} \cdot \overline{Q_1} \cdot \overline{Q_0}$ (Detects when $Q = 000$)
- **Equality Comparator:** $E = \overline{x_2} \cdot \overline{x_1} \cdot \overline{x_0}$ where $x_i = Q_i \oplus R_i$ for $i = 0, 1, 2$

Logic Derivation

To determine the next state (U_{next}) and the immediate control signal for the counter, we analyze the transitions:

Current State (U)	Inputs (E, Z)	Logic	Next State (U_{next})	Action
1 (UP)	$E = 0$	Keep going up	1	Increment
1 (UP)	$E = 1$	Limit reached (B)	0	Decrement
0 (DOWN)	$Z = 0$	Keep going down	0	Decrement
0 (DOWN)	$Z = 1$	Bottom reached (0)	1	Increment

Next State Equation:

We need $U_{next} = 1$ (next action is Count Up) when:

1. We are currently UP ($U = 1$) AND we have NOT reached the limit ($\bar{E}$).
2. OR we are currently DOWN ($U = 0$) AND we have hit zero (Z).

$$U_{next} = (U \cdot \bar{E}) + (\bar{U} \cdot Z)$$

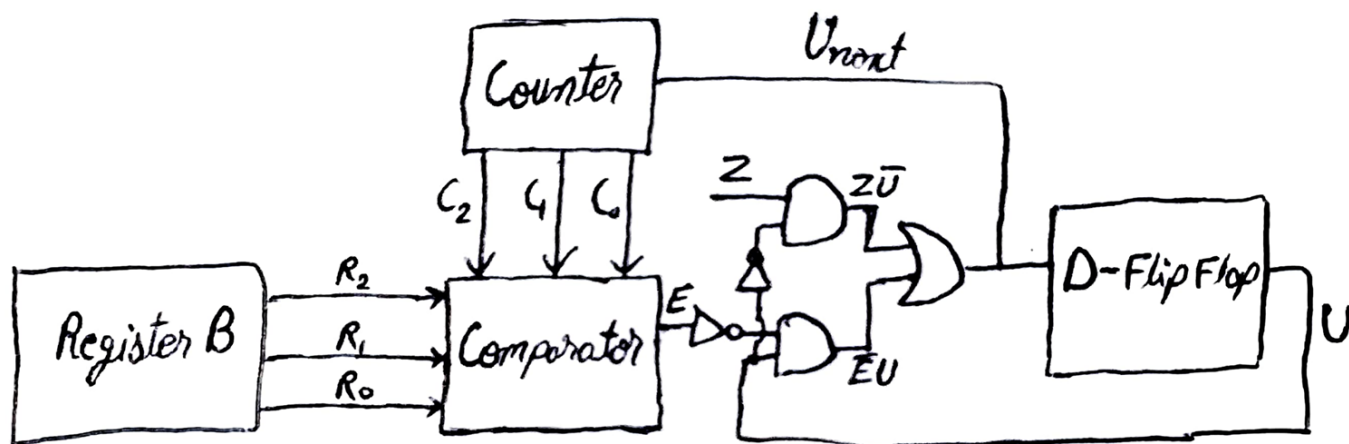
Critical Design Note (Look-Ahead Logic): If we were to connect the flip-flop output (U) directly to the counter's control pin, the counter would only reverse direction *after* the next clock edge, causing it to overshoot to $B + 1$. To prevent this, we connect the combinational signal U_{next} directly to the counter. This acts as a "look-ahead" signal, ensuring the counter receives the command to reverse direction immediately within the same clock cycle that the limit is reached.

Eventual Circuit Design

The circuit consists of the Control Logic (derived above) and the Datapath (Black boxes).

- **Control Signal:** The U_{next} signal is fed to the D-Flip Flop (to update state for next clock) AND directly to the Counter's Mode pin (to change direction immediately) as explained above in look ahead logic.
- **Datapath Components:**
 - **Register B:** Holds the 3-bit limit value.

- **Comparator:** Compares Counter Q with Register B to generate E .
- **Counter:** 3-bit synchronous up/down counter, controlled by U_{next} .
- **Zero Detector:** Checks if Counter Q is zero to generate Z .
- **State Flip-Flop:** D-Flip Flop to store current state U .
- **Logic Gates:** AND and OR gates to implement the U_{next} logic.



Synchronous 3-bit UP/DOWN Counter Circuit

The Circuit Diagram for conditions E and Z can be drawn using the following logic expressions:

$$E = \overline{(Q_2 \oplus R_2)} \cdot \overline{(Q_1 \oplus R_1)} \cdot \overline{(Q_0 \oplus R_0)}$$

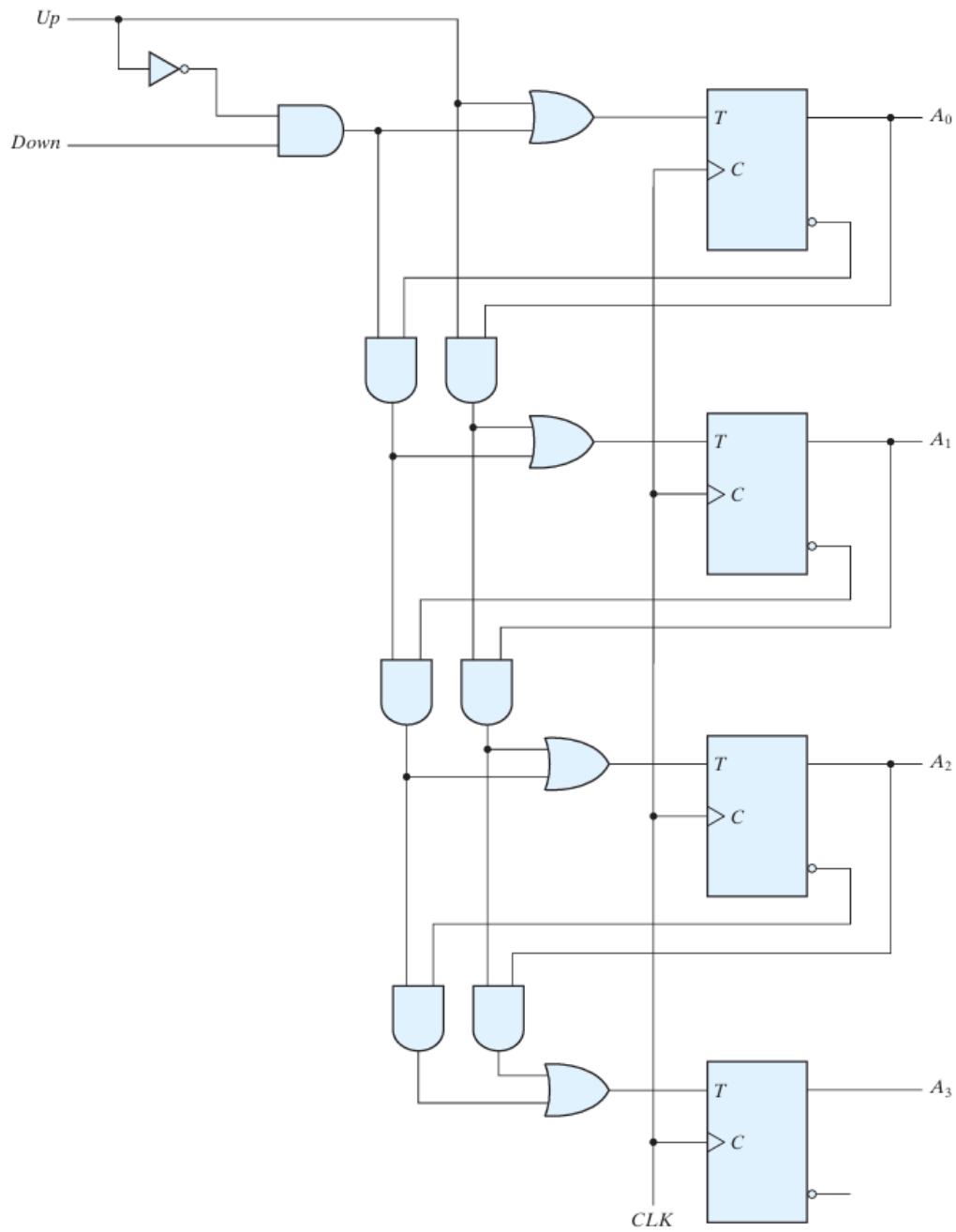
$$Z = \overline{Q_2} \cdot \overline{Q_1} \cdot \overline{Q_0}$$

U_{next} is same as Up input shown in the Counter circuit diagram below. But Down is not simply the inverse of Up , because of the following edge case:

Edge Case (Register $B = 0$): When $B = 0$, the counter will keep toggling between 0 and 1 continuously. So, to avoid that, we can add an additional AND gate to force both Up and $Down$ to 0 when $B=0$. Therefore,

$$Up = U_{next} \cdot (R_2 + R_1 + R_0)$$

$$Down = \overline{U_{next}} \cdot (R_2 + R_1 + R_0)$$



Four-bit up-down Binary Counter

Question-4

Design a sequential circuit that detects if the number of ones in a four bit bitstream is even or odd. The bitstream input is serial. After four bits have appeared serially the circuit resets to evaluate the next sequence of four serial bits and this continues perpetually. Represent this as a FSM clearly describing the states, inputs and outputs. Eventual circuit design is not expected. [10 marks]

Solution:

State Notation Legend

States are labeled using the format: P_n/Z

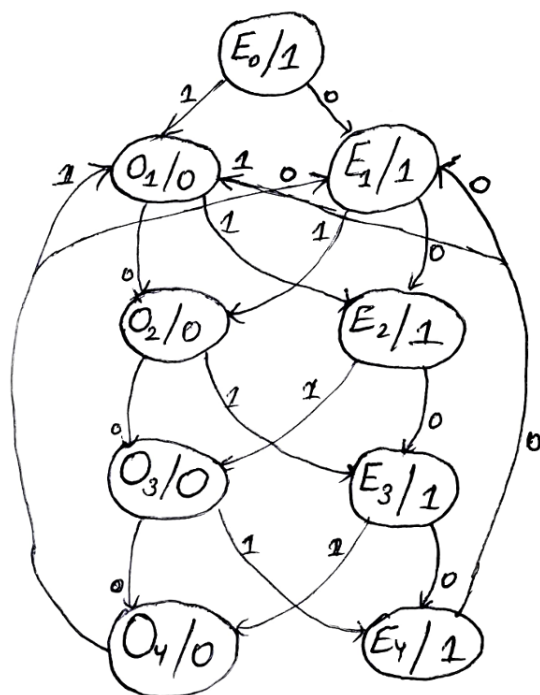
- **P (Parity):** Indicates the parity of ones counted so far in the current sequence
 - E = Even Parity (even number of ones seen)
 - O = Odd Parity (odd number of ones seen)
- **n (Bit Count):** Number of bits processed in current 4-bit sequence (0, 1, 2, 3, or 4)
- **Z (Output):** Logic output associated with the state
 - 1 = Total ones is Even
 - 0 = Total ones is Odd

State Transition Logic

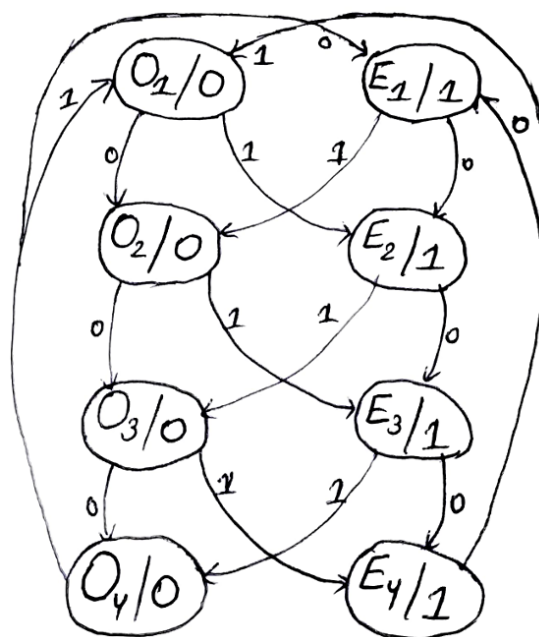
Transitions follow these rules for serial input X :

- **If Input $X = 0$:** Number of ones does not change
 - Current State (Even) $\rightarrow$ Next State (Even)
 - Current State (Odd) $\rightarrow$ Next State (Odd)
- **If Input $X = 1$:** Number of ones increases by 1, flipping the parity
 - Current State (Even) $\rightarrow$ Next State (Odd)
 - Current State (Odd) $\rightarrow$ Next State (Even)

State Diagram for Even/Odd Parity Detection



State Diagram (For Understanding)



Final Reduced State Diagram

To get started designing the FSM, we first define the states based on the number of bits processed (0 to 4) and their parity (Even or Odd) as discussed on the previous page. You might think, how will you get the idea of taking these states from the start. But the beauty of this state diagram is that even if you start with whatever states (extra states) you think are necessary, through systematic analysis and state reduction (merging equivalent states), you will naturally arrive at the optimized state diagram shown above.

Starting from the initial state $E_0/1$ (0 bits processed), we analyze the transitions for each input bit (0 or 1) and update the state accordingly. For example, after receiving the first bit: we can have either $E_1/1$ (if the bit is 0) or $O_1/0$ (if the bit is 1). Then from the state $E_1/1$, receiving the second bit 0 keeps us in $E_2/1$, while receiving 1 takes us to $O_2/0$. Continuing this process for all four bits, we build out the complete state diagram as shown in the left figure above.

After processing 4 bits, the FSM resets from $E_4/1$ or $O_4/0$ to $E_1/1$ or $O_1/0$ (Not to the initial state, $E_0/1$) to evaluate the next sequence. Transitions from Level 4 (O_4 , E_4) go directly to Level 1 (O_1 , E_1) as the 4th bit displays the result while the next clock cycle captures the 1st bit of new sequence. Here, we merge the states E_0 and E_4 since they both have same outputs and next states, resulting in the final reduced state diagram shown on the right.

Note:

- The key is to ensure that after processing 4 bits, the FSM transitions back to the appropriate state to start evaluating the next sequence.

Question-5

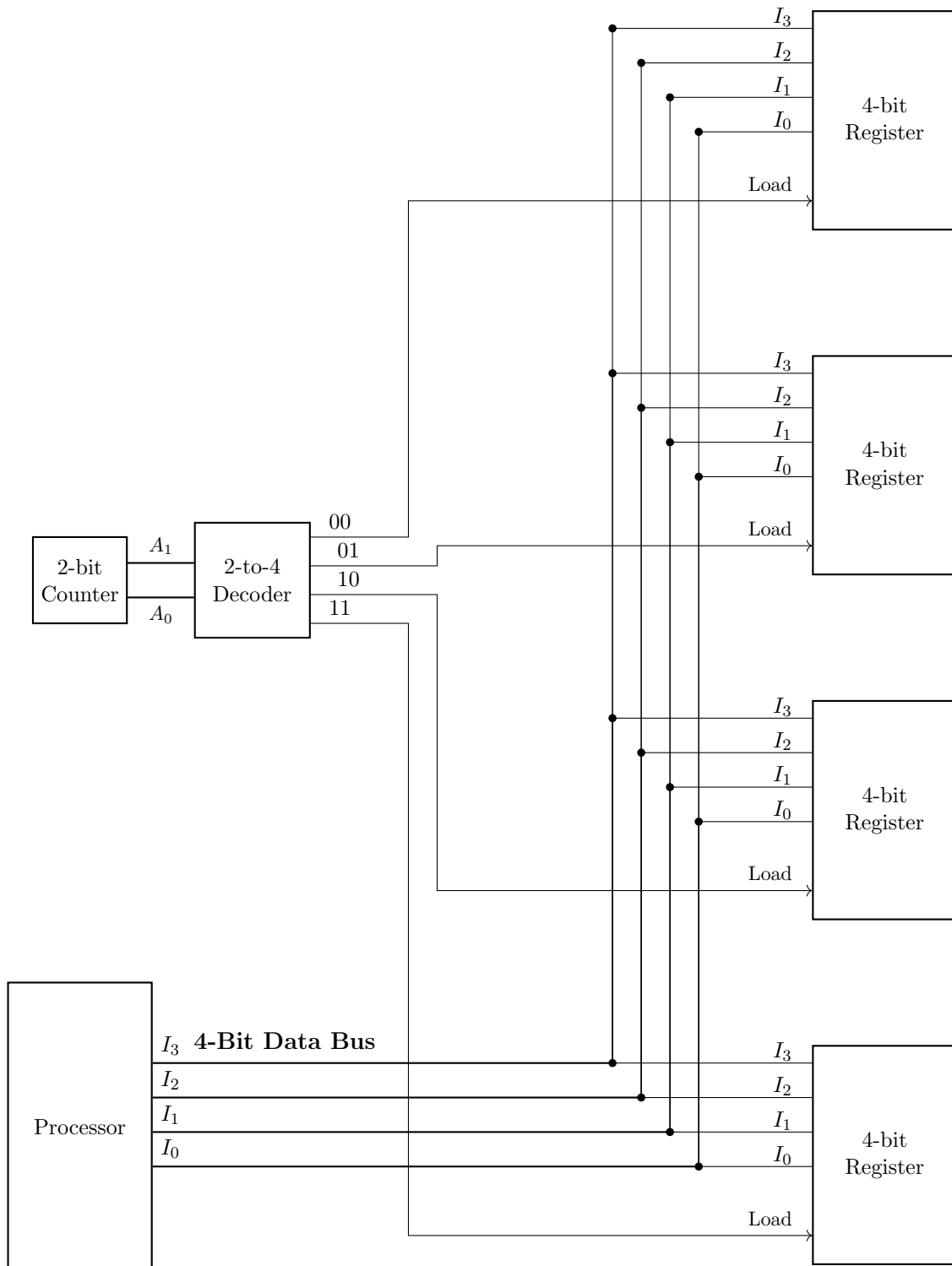
Consider a processor represented by a black box with four data lines that come out of it. Each of these lines carries voltage logic levels indicative of 1 or 0, which would need to be stored/ loaded into appropriate 4 bit registers. Assume there are four such registers, whose address varies from 00 to 11. Assume also that there is a 2 bit binary counter. Design a circuit that loads the contents of the data bus onto that register whose address is given by the current counter value. Assume the counter to be a black box with its outputs readily available. Your design should be illustrated as a combination of/ interconnection between various functional blocks. [10 marks]

Solution:

Following are the main components of the design:

- **2-bit Counter:** The 2-bit counter acts as the address generator. To select a specific register, generates a 2-bit address (A_1, A_0) that cycles through 00, 01, 10, and 11 which must be translated into a unique enable signal.
- **2-to-4 Decoder/ 1-to-4 Demultiplexer:** Takes the 2-bit address from the counter and activates one of its four outputs (E1, E2, E3, E4) corresponding to the address.
- **4-bit Data Bus:** Carries the 4-bit data (I3, I2, I1, I0) from the processor to be loaded into the selected register.
- **Four 4-bit Registers:** Each register can load data from the data bus when its corresponding enable signal (E1 to E4) is activated by the decoder.

Diagram on next page represents a simple architecture where the data bus load its content into the 4-bit register selected by the 2-bit address line address decoded by the 2:4 decoder.



Question-6

- | | |
|---|-----------|
| (a) Explain what is ROM & its use. | [2 marks] |
| (b) How can it be programmed to store certain values? | [2 marks] |
| (c) For 3 bit input (A), design a circuit using ROM, which will give you output $O = 4 * A - 1$. | [6 marks] |

Solution:

(a) What is ROM & its Use

1. **Definition:** ROM stands for Read-Only Memory. It is a type of non-volatile memory used in computers and other electronic devices. "Non-volatile" means it retains its stored data even when the power is turned off. Data stored in ROM can only be read; it cannot be easily or quickly modified (written to).
2. **Uses:**
 - **Combinational Circuit Implementation:** ROM can be used to implement any desired combinational circuit by storing the truth table. The ROM essentially acts as a decoder and OR gates combined.
 - **Permanent Storage:** Store fixed system data such as firmware (BIOS/-Bootstrap loaders), lookup tables, conversion tables
 - **Code Conversion:** Binary-to-BCD, ASCII character generation, etc.

(b) How ROM Can Be Programmed

Depending on the specific type of ROM, it can be programmed to store values in four main ways:

- **Mask Programming (Factory):** The binary pattern is permanently hard-wired during the semiconductor fabrication process based on a truth table provided by the customer. This can not be altered later.
- **Blowing Fuses (PROM):** The user creates the pattern by applying high voltage pulses to "blow" (break) specific internal fuses, changing a logic 1 to 0 (or vice versa).
- **EPROM (Ultraviolet Erasure):** Uses floating-gate transistors where data is stored by trapping electrical charges using high voltage. To erase the data, the chip is exposed to special Ultraviolet (UV) light, which discharges the gates so the chip can be reprogrammed.
- **EEPROM (Electrical Erasure):** Also uses charge storage but allows erasing and reprogramming electrically without removing the chip from the circuit.

Unlike EPROM, it does not require UV light, it can be erased using electrical signals. This is the technology used in Flash memory (USB drives, SSDs).

(c) ROM Design for $O = 4 \times A - 1$

Note: Solution for $O = 4 \times A + 1$ is also accepted

If the problem is $O = 4A + 1$ instead:

- Output range: $A = 0 \rightarrow O = 1$, $A = 7 \rightarrow O = 29$ (all positive, no negative values)
- ROM size: 8×5 (only 5 bits needed since no 2's complement required; max value $29 = 11101_2$)
- Truth table: Simply compute $4A + 1$ for each input (e.g., $A = 0 : O = 1 = 00001$, $A = 7 : O = 29 = 11101$)
- Pattern: $O_0 = 1$ (always), $O_1 = 0$ (always), $O_2 = A_0$, upper bits vary
- Optimization: Store only O_4, O_3 in 8×2 ROM; others are direct: $O_2 = A_0$, $O_1 = 0$, $O_0 = 1$

The solution methodology remains identical; only truth table values and bit width change.

Step 1: Determine ROM Size

Input Analysis:

- Input: 3-bit number $A = A_2A_1A_0$ (ranges from 0 to 7)
- Number of address lines: $k = 3$
- Number of addressable locations: $2^3 = 8$ words

Output Analysis:

- For $A = 0$: $O = 4 \times 0 - 1 = -1$ (in 2's complement representation)
- For $A = 7$: $O = 4 \times 7 - 1 = 27$ (requires 5 bits for unsigned, but we need signed representation)
- Maximum output value: 27 (binary: 011011), minimum: -1 (binary: 111111 in 6-bit 2's complement)
- Since we have negative values, we need signed representation (2's complement)
- Number of output bits required: $n = 6$ bits ($O = O_5O_4O_3O_2O_1O_0$) to represent range $[-1, 27]$ in 2's complement

ROM Configuration: 8×6 ROM (8 words, 6 bits per word)

Step 2: Derive Truth Table

The first step is to derive the truth table of the combinational circuit. The truth table for the ROM is derived by computing $O = 4A - 1$ for each input value:

Inputs (Address)					Outputs (Data)						Notes
A_2	A_1	A_0	Dec A	Dec O	O_5	O_4	O_3	O_2	O_1	O_0	
0	0	0	0	-1	1	1	1	1	1	1	2's comp: 111111
0	0	1	1	3	0	0	0	0	1	1	
0	1	0	2	7	0	0	0	1	1	1	
0	1	1	3	11	0	0	1	0	1	1	
1	0	0	4	15	0	0	1	1	1	1	
1	0	1	5	19	0	1	0	0	1	1	
1	1	0	6	23	0	1	0	1	1	1	
1	1	1	7	27	0	1	1	0	1	1	

Table 2: Truth Table for Circuit implementing $O = 4A - 1$

Note: For $A = 0$, the output is -1 . In 6-bit 2's complement representation, -1 is represented as 111111. All positive outputs have MSB (O_5) = 0.

Alternative: Truth Table for $O = 4A + 1$ (5-bit output, no 2's complement needed)

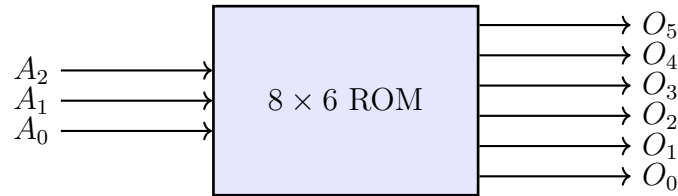
Inputs (Address)					Outputs (Data)					Notes
A_2	A_1	A_0	Dec A	Dec O	O_4	O_3	O_2	O_1	O_0	
0	0	0	0	1	0	0	0	0	1	
0	0	1	1	5	0	0	1	0	1	
0	1	0	2	9	0	1	0	0	1	
0	1	1	3	13	0	1	1	0	1	
1	0	0	4	17	1	0	0	0	1	
1	0	1	5	21	1	0	1	0	1	
1	1	0	6	25	1	1	0	0	1	
1	1	1	7	29	1	1	1	0	1	Max: $29 = 11101_2$

Table 3: Truth Table for Circuit implementing $O = 4A + 1$ (ROM size: 8×5)

Note: For $O = 4A + 1$, all outputs are positive (range 1 to 29), so only 5 bits are needed. Observe: $O_0 = 1$ (always), $O_1 = 0$ (always), $O_2 = A_0$ (direct copy).

Step 3: ROM Block Diagram

The ROM is specified by the truth table information shown above. The three inputs specify eight words (addresses 0-7), and the six outputs give the information for the six bits in each word. The ROM size required is 8×6 (8 words of 6 bits each).



ROM Block Diagram for $O = 4A - 1$

ROM Truth Table Representation:

The ROM can also be represented by showing the stored word content at each address:

Address ($A_2A_1A_0$)	ROM Content (Output)					
	O_5	O_4	O_3	O_2	O_1	O_0
0	1	1	1	1	1	1
1	0	0	0	0	1	1
2	0	0	0	1	1	1
3	0	0	1	0	1	1
4	0	0	1	1	1	1
5	0	1	0	0	1	1
6	0	1	0	1	1	1
7	0	1	1	0	1	1

Table 4: ROM Truth Table: Content stored at each address

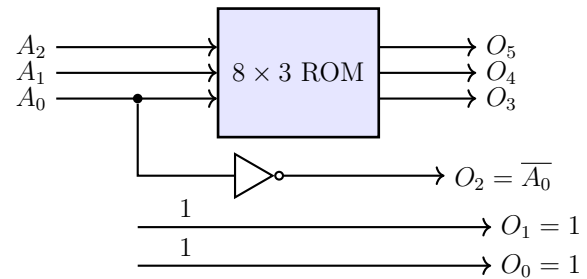
Optimization

By examining the ROM truth table carefully, we can observe that some output bits follow a simple pattern:

- O_0 (**LSB**): Always equals 1 for all addresses (constant output)
- O_1 : Always equals 1 for all addresses (constant output)
- O_2 : Equals $\overline{A_0}$ (complement of input bit A_0)

Since these three outputs can be generated using simple logic (constants and direct input connection), we don't need to store them in ROM. We can reduce the ROM size by storing only the remaining three outputs (O_5, O_4, O_3) in ROM.

Address ($A_2A_1A_0$)	ROM Content		
	O_5	O_4	O_3
0	1	1	1
1	0	0	0
2	0	0	0
3	0	0	1
4	0	0	1
5	0	1	0
6	0	1	0
7	0	1	1



Optimized Circuit using 8×3 ROM
with direct connections for O_2, O_1, O_0

Optimized ROM Truth Table: Only
 O_5, O_4, O_3 stored

Hardware Savings:

- Original design: 8×6 ROM = 48 memory cells
- Optimized design: 8×3 ROM = 24 memory cells + simple wiring
- **Reduction: 50% reduction in ROM size**

Note: The optimization takes advantage of the fact that O_0 and O_1 are always 1, and $O_2 = \overline{A_0}$ for all input combinations. This is because in the expression $O = 4A - 1$, the two LSBs follow a predictable pattern.

Question-7

Explain the difference between operations **Jmpd** and **Cd**. What are the differences in the micro-instructions cycle and why? [2 marks]

Explain using '**cdu 110**', what micro-instructions are needed in each microcycle? Show the code (as if stored in the memory) along with proper values of PC, SP for each step. [4 marks]

Show timing diagram for its fetch/execute cycle (4 clock cycle). Assume, $[SP] = 250$ and $[PC] = 02$ at the starting. S is a Sign flag. [4 marks]

Solution:

(a) *Difference between Jmpd and Cd*

1. **Operations:** **Jmpd** (Jump Direct) simply updates the Program Counter (PC) to a new address, effectively branching to a new location. **Cd** (Call Direct) is used for function calls; it first pushes the current PC (return address) onto the Stack and then updates the PC to the new address.
2. **Micro-instructions cycle:**
 - **Jmpd** takes 4 cycles total (2 fetch + 2 execute) whereas **Cd** takes 7 cycles total (2 fetch + 5 execute). **Jmpd** only needs to read the new address and load it into the PC to jump but **Cd** requires extra micro-cycles to decrement the Stack Pointer (D_{SP}) and write the return address to memory (WR) before jumping.

Part (b): Micro-instructions for 'cdu 110'

1. Assumptions & Initial State:

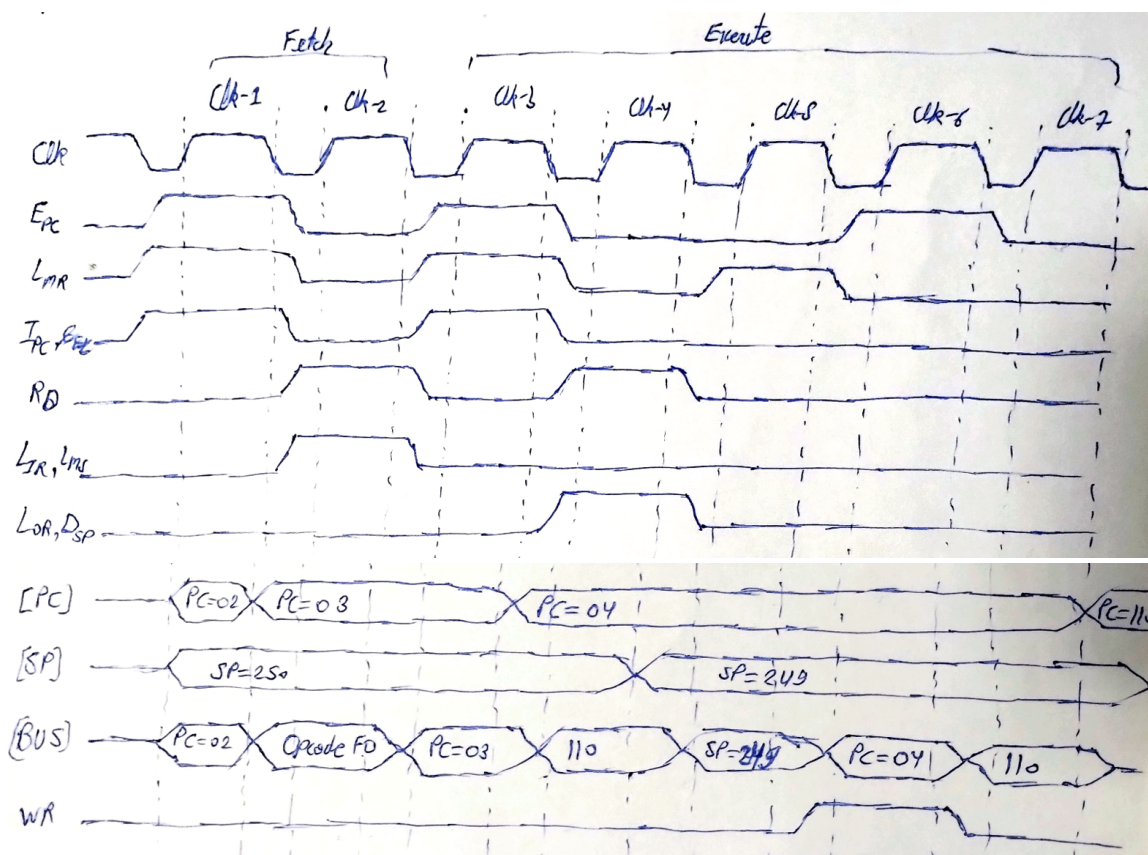
- **Initial Values:** $[PC] = 02$ and $[SP] = 250$ (as given).
- **Opcode:** From the instruction set, the opcode for **cdu** (Call Direct Unconditional) is **F0**.
- **Timing:** The instruction requires **7 clock cycles** total (2 for fetch + 5 for execution).
- **Return Address:** Since **cdu 110** occupies 2 memory words (Addresses 02 and 03), the return address saved to the stack must be the *next* instruction address, which is **04**.

Clk	Micro-instructions	Action / Comment	[PC]	[SP]
<i>Phase 1: Instruction Fetch</i>				
1	E_{PC}, L_{MR}, I_{PC}	Move PC(02) to MAR; Inc PC to 03	03	250
2	RD, L_{IR}, L_{MS}	Read Opcode (F0) into IR; Decode	03	250
<i>Phase 2: Operand Fetch & Stack Preparation</i>				
3	$E_{PC}, L_{MR}, I_{PC}, E_{FL}$, End if $< FL >'$	Addr of Imm(03) to MAR; PC becomes 04 (Ret Addr)	04	250
4	RD, L_{OR}, D_{SP}	Read Imm Data (110) to OR; Dec SP	04	249
<i>Phase 3: Saving Return Address (Push)</i>				
5	E_{SP}, L_{MR}	Move SP(249) to MAR for write	04	249
6	E_{PC}, WR	Write PC(04) to Stack Memory at [249]	04	249
<i>Phase 4: Execution Jump</i>				
7	E_{OR}, L_{PC}, End	Load Jump Addr (110) from OR to PC	110	249

Table 5: Cycle-by-cycle analysis of `cdu 110`

Part (c): Timing Diagram for Fetch/Execute Cycle

Given: Initial $[SP] = 250$, $[PC] = 02$



Note: The above timing diagram shows all 7 cycles, but only 4 clock cycles will be considered for marking as per the question requirement.

Question-8

Explain in short (to the point):

- (a) SR latch & working [2 marks]
- (b) Universal Shift Register [2 marks]
- (c) Mealy vs Moore state machines [2 marks]
- (d) Which is better for synchronous design & why: edge triggered vs level triggered. [2+2 marks]

Solution:

(a) SR Latch & Working

1. **Definition:** An SR (Set-Reset) latch is a fundamental asynchronous sequential circuit with two inputs (S and R) and two complementary outputs (Q and $\bar{Q}$). It is the most basic memory element in digital systems.
2. **Working Principle (or Truth Table):**
 - **Set (S=1, R=0):** Sets the output Q to 1 and $\bar{Q}$ to 0.
 - **Reset (S=0, R=1):** Resets the output Q to 0 and $\bar{Q}$ to 1.
 - **Hold (S=0, R=0):** Maintains the previous state (memory action).
 - **Forbidden (S=1, R=1):** Creates an invalid state where both outputs attempt to be 0, leading to unpredictable behavior when inputs change.
3. **Hardware Implementation:** Typically built using cross-coupled NOR gates or NAND gates. The NOR-based SR latch is active-high, while the NAND-based version is active-low (also called $\overline{SR}$ latch).

Note: For grading purposes, any **two** of the above three points are sufficient for full marks.

(b) Universal Shift Register

1. **Definition:** A Universal Shift Register is a versatile register that can perform multiple operations based on control signals. It combines the functionality of parallel load, serial shift (left/right), and parallel output in a single unit.
2. **Key Operations:**
 - **Parallel Load:** Load all bits simultaneously from parallel inputs.
 - **Shift Right:** Serial shift data from MSB to LSB.

- **Shift Left:** Serial shift data from LSB to MSB.
 - **Hold:** Retain current data without any change.
3. **Control Mechanism & Applications:** Typically uses 2-bit mode select lines (S1, S0) to determine the operation: S1S0 = 00 (Hold), 01 (Shift Right), 10 (Shift Left), 11 (Parallel Load). Applications include serial-to-parallel and parallel-to-serial data conversion, arithmetic operations, and data storage/manipulation.

Note: For grading purposes, any **two** of the above three points are sufficient for full marks.

(c) Mealy vs Moore State Machines

Aspect		Mealy Machine	Moore Machine
Output Dependency	Depen-	Output depends on both current state and current inputs . $Output = f(State, Input)$	Output depends only on the current state . $Output = f(State)$
Response Time		Faster response - outputs change immediately with input changes (asynchronous to state transitions).	Slower response - outputs change only after state transitions (one clock cycle delay).
Number of States		Generally requires fewer states for the same functionality.	May require more states to encode the same behavior.
Output Stability		Outputs may have glitches or transient changes due to direct input dependency. Susceptible to noise.	Glitch-free, stable outputs synchronized with clock edges. More predictable and reliable.

Note: For grading purposes, any **two** of the above four distinguishing aspects are sufficient for full marks.

(d) Edge Triggered vs Level Triggered for Synchronous Design

Edge-triggered flip-flops are better for synchronous design because:

1. **Edge Triggered Flip-Flops** respond only at clock transitions (rising or falling edge), where data is sampled and latched at the specific clock edge, while **Level**

Triggered Latches are transparent when clock is at active level (high or low), with output following input during the entire active clock period. This makes edge-triggered flip-flops provide deterministic, well-defined sampling points.

2. Edge-triggered flip-flops eliminate race conditions and timing ambiguities inherent in level-triggered designs. Level-triggered latches are susceptible to race conditions and timing hazards, where clock skew can cause multiple unwanted state changes.
3. Edge-triggered flip-flops enable reliable cascading and pipelining of sequential stages without timing issues, while it is difficult to cascade level-triggered latches without careful timing analysis.
4. Edge-triggered flip-flops provide better noise immunity and reduced power consumption. They are immune to clock pulse width variations and noise during the clock level.
5. Edge-triggered flip-flops are less sensitive to clock duty cycle and pulse width variations, enabling precise timing control in synchronous systems.

Note: For the "why" part (2 marks), any of the above five points are sufficient for full marks.